altairsim — Sol-20 cassette games

2026-09-06 · 4d137d1

A Processor Technology **Sol-20** with a 1977 game in the cassette deck. Four of them, each in its own machine file — pick one and run it:

```
altairsim trek80.toml        # TREK80  -- Star Trek
altairsim atc.toml           # ATC     -- Air Traffic Controller
altairsim pacman.toml        # PAC-MAN
altairsim raiders.toml       # RAIDERS
```

The game loads itself. Each file mounts the cassette, types the SOLOS command that runs it, and boots — so after the tape finishes loading (about the minute a real cassette took, at the Sol's own speed) the game comes up. TREK80 asks for a speed factor and starts; the others run straight away.

You do not have to type anything — but on a real Sol-20 you would, and the machine files simply do it for you. At the SOLOS `>` prompt you would enter `XE` and the file's five-character name; a `TYPE` line in each `startup` queues exactly that, and SOLOS reads it as type-ahead at its first prompt, as if the keys had been pressed. The names, for when you `XE` one yourself after a `REW`:

Game	Command
TREK80	`XE TRK80`
ATC	`XE ATC`
PAC-MAN	`XE PACMA` — SOLOS names are five characters
RAIDERS	`XE RAID`

What you are looking at

The Sol-20 is not an Altair with a terminal on it — it is an integrated computer with a **keyboard and a screen built in**, and that changes what this looks like on your terminal. The screen is a **VDM-1**, memory-mapped video at `0CC00H`–`0CFFFH`, and nothing it displays reaches your terminal: SOLOS signs on, `XE` echoes, and the game paints its starfield all into that video memory. With SDL3 you get a **window**, in the VDM-1's own character font, and you watch it there. Built without SDL the machine still runs perfectly and the console stays quiet — that quiet is correct. Your typing goes the other way and does work: keystrokes reach the Sol's keyboard port.

Headless, you can still read what the game painted: stop the machine with `^E` and dump the VDM's memory. Each row is 64 bytes, and `DUMP … WIDTH=64` prints the ASCII beside the hex, so one screen row is

one line. For TREK80:

```
altairsim> DUMP CD00-CD3F WIDTH=64
CD00  43 4F 50 59 52 49 47 48 54 ... 43 4F 52 50 2E  COPYRIGHT (C) 1977  PROCESSOR TECHNOLOGY CORP.

altairsim> DUMP CFC0-CFFF WIDTH=64
CFC0  45 4E 54 45 52 20 53 50 ... 54 29 29           ENTER SPEED FACTOR (9(SLOW)-0(FAST))
```

That is the game's own copyright line off a 1977 tape, and the row where it stops to ask you something — read straight out of the screen it drew them on. `RUN` resumes, and the answer you type reaches the game. `^E` (STOP) takes the keyboard back to the monitor at any point; the machine is not disturbed by stopping it.

The clock is 2.045 MHz, the stock Sol-20 — the 14.31818 MHz dot clock divided by 7 (Sol Systems Manual, Theory of Operation §VIII). Everywhere else in this simulator the default is flat out; here the real speed is set on purpose, because these are games and they were played at that speed. It is also why the tape takes a real minute to load. `SET cpu0 clock_hz=0` puts it back to flat out.

To play a game a second time, rewind first — nothing inside the guest can wind a tape back:

```
REW sol0:tape1
XE TRK80
```

The files

File	What it is
`trek80.toml` · `atc.toml` · `pacman.toml` · `raiders.toml`	The four machines: `base = "sol20"`, a tape, the launch keystrokes, and the clock.
`TRK80.WAV` · `ATC.WAV` · `PACMAN.WAV` · `RAIDERS.WAV`	**The tapes.** Sol-20 cassettes digitized by Philip Lord (hosted on deramp.com) — real CUTS audio at 1200 baud, decoded the way the hardware did.
`TRK80.TAP`	TREK80 as a byte stream, decoded from `TRK80.WAV`. `MOUNT sol0:tape1 "TRK80.TAP"` skips the audio and loads the same game.
`Trek80 Manual.pdf` · `ATC Manual.pdf`	The game manuals — commands, displays, and scoring — from Processor Technology and Creative Computing.
`TREK80.ENT`	A SOLOS `ENTER` script from the archive. The source for the tape-writing demonstration below.
`make-trek80-tape.sh`	A demonstration: it has SOLOS write its own tape from `TREK80.ENT`.

The tapes are Philip Lord's real recordings, and reading them is the point. Each `.WAV` decodes to a valid SOLOS tape with **zero framing errors** — for TREK80 the header names `TRK80`, its `SIZE` is `1EA0`,

and its checksum is `D9`, all matching the archived `ENTER` script byte for byte. That did not used to work: an earlier decoder garbled these exact recordings (dozens of framing errors, most of the payload wrong), and the TREK80 example tape had to be *synthesized* from the `.ENT` instead. The demodulator now reads the real cassettes, so the real cassettes are what ship.

The synthesis is kept as `make-trek80-tape.sh` because it demonstrates the one thing a SOLOS tape cannot fake: a SOLOS cassette carries a header checksum and one after every 256-byte block, and a tape whose checksums are wrong is *invisible* — `CA` lists nothing and `GE` never finds the file. The script has SOLOS `SA`ve the image so the checksums are the machine's own arithmetic; it writes to a separate `TRK80-solos.*` so it never overwrites the shipped recording, and it proves the two agree — SOLOS computes the same `D9` header checksum the real tape carries.

See `docs/sources.md` for full provenance.